

# Whole-Protocol Admission for Repurposed Ray-Tracing Programs

Anonymous Author(s)

## Abstract

Ray-tracing APIs validate program and platform legality but leave application meaning distributed across host composition, multiple device callbacks, payload and attribute conventions, continuation code, and the executable that is ultimately launched. We present RTDL, a bounded whole-protocol compilation and admission model for repurposed OptiX computations. RTDL checks cross-role effects, semantic ABI ownership, physical bindings, status-gated continuation and completeness, and executable identity, then exposes a typed lifecycle for exactly two closed public protocol families.

On two project-designed non-rendering tasks, single-leaf mutations holding compiler projection bytes fixed show that all 19 populated contract leaves are decision-bearing: 7 role/effect, 1 semantic-ABI, 7 physical-binding, 2 continuation/completeness, and 2 executable-identity leaves. This establishes liveness for these two contracts, not vocabulary completeness, target-projection correctness, or a soundness theorem. In a disclosed two-line repaired PyOptiX FFI route with OptiX 9 typed-payload semantics active, all five pre-registered fault realizations reach launch; a separate pinned NVIDIA OWL route also launches all five. The supported responsibility result is three full wrong-output residuals, one partial status-gated continuation/completeness residual, and checked-executable binding support; RTDL rejects the corresponding declarations before launch.

Across nine project-authored ports of published RT mappings, 13 selected lanes execute exactly through the base callback path and six application-neutral composition batches. One project-selected SQL bag equijoin reuses an existing family without a core change, but prospective unbiased new-application exams, third-party authors, and human usability studies remain zero. For performance, we compare current RTDL with Direct CUDA/OptiX and PyOptiX-compatible, a pinned NVIDIA `otk-pyoptix` 9.1.0 source build using the OptiX 9.0 compatibility API. On two matched tasks on one RTX 4000 Ada, steady RTDL/PyOptiX-compatible passes the registered 1.05 noninferiority gate, with median differences of  $-17\ \mu\text{s}$  and  $+1.22\ \mu\text{s}$ , while post-import deployment-cold and prepare show setup deltas of 162–223 ms. The Direct arm is descriptive. RTDL does not discover ray-tracing mappings, execute arbitrary Callback IR, prove accepted-program correctness, or establish usability superiority or universal performance parity.

CGO '27, Salt Lake City, UT, USA  
2027.

**CCS Concepts:** • Software and its engineering → Domain specific languages; Software verification; • Computing methodologies → Ray tracing.

**Keywords:** ray tracing, domain-specific languages, protocol admission, application-semantic contracts, OptiX

## ACM Reference Format:

Anonymous Author(s). 2027. Whole-Protocol Admission for Repurposed Ray-Tracing Programs. In *Proceedings of IEEE/ACM International Symposium on Code Generation and Optimization (CGO '27)*. ACM, New York, NY, USA, 14 pages.

## 1 Introduction

Hardware ray-tracing APIs are increasingly repurposed for non-rendering computations such as spatial queries, graph processing, databases, and simulation. A recent review identifies 59 papers and systematically analyzes 35 proposed RT solutions spanning 32 distinct problems, while also observing that profitable mappings remain irregular and difficult to predict [26].

For the application-to-RT correspondence studied here, we organize relevant systems by three established responsibilities. *Interface systems* validate shader-stage interfaces, access contracts, and target legality [19, 27, 33, 37, 45]. *Construction systems* expose or organize host bindings, shader modules, acceleration structures, pipelines, shader binding tables (SBTs), resources, and launches [13, 14, 35, 36]. *Mapping systems* trace, specialize, or translate higher-level computations to RT-capable backends [7, 18, 55]. Systems may span classes; these labels identify their primary responsibility rather than their implementation form.

Even when one system spans these responsibilities, the documented checking units above do not jointly admit a repurposed computation as one application-semantic protocol. Its meaning is distributed across multiple device roles, payload and attribute slots, geometry construction, pipeline and SBT bindings, device status, host continuation, and the executable bytes that finally launch. Each piece may be legal while the complete protocol is wrong. A hit program can interpret an attribute as a primitive index although the intersection program wrote an application item identifier; a callback can omit state required by a downstream role; a host can consume a capacity-truncated output despite an overflow status; or a reviewed declaration can be paired with a different executable. Such faults are particularly dangerous in non-rendering workloads, where application-level exactness and completeness invariants are not represented by the RT API contract.

This paper studies a fourth responsibility, *protocol admission*, and asks a deliberately narrower question than automatic mapping: *after a developer has chosen a traversal formulation, can the complete callback protocol become the unit of compilation and admission?* We present RTDL, which admits two closed public protocol families. The source and physical plan are checked as one obligation set; an accepted program is bound to generated code, target and native identities; and execution is exposed through a typed materialize–prepare–execute–close lifecycle. The application continues to own its algorithm, semantic oracle, and the decision to use ray tracing.

This bounded framing is important. RTDL is not OptiX-equivalent, does not execute arbitrary verified Callback IR on the public GPU path, and does not establish prospective or third-party generalization. Its value proposition is instead analogous to a protocol IDL: it does not simplify the application algorithm, but it makes selected cross-component mismatches unrepresentable or rejectable before launch.

This paper contributes:

- a protocol-admission compilation unit, distinct from interface checking, construction, and mapping, that checks cross-role effects, semantic ABI ownership, physical bindings, status-gated continuation and completeness, and checked-executable identity as one unit, and carries acceptance through a typed materialize–prepare–execute–close lifecycle;
- a bounded OptiX implementation with exactly two closed public GPU protocol families, exercised by nine project-authored ports over thirteen selected paper-derived lanes and six application-neutral composition batches; this is not an arbitrary Callback-IR-to-GPU compiler; and
- a claim-calibrated evaluation: all 19 populated leaves in two contracts are decision-bearing, with seven, one, seven, two, and two leaves for CP001–CP005, respectively; active OptiX typed-payload and pinned OWL executions support three full wrong-output residuals, one partial status-gated continuation/completeness residual, and executable-binding support; one project-selected reuse and one failed transfer are preserved; and a same-target three-arm successor reports both the passing steady and failing setup results against PyOptiX-compatible, with Direct CUDA/OptiX descriptive.

The leaf sweep is liveness evidence for two populated contracts. It does not establish vocabulary completeness, target-projection correctness, semantic soundness, or correctness of arbitrary accepted applications.

## 2 Motivating Failure: Asking “Which Item?” and Getting “Which Shelf?”

**One-sentence version.** RTDL prevents a repurposed RT program from silently returning *where an object is stored* when the application asked *which object it is*.

Imagine a warehouse with shelf 0 holding item 10 and shelf 1 holding item 20. Lookup 100 should return item 10, and lookup 101 should return item 20. A buggy worker instead returns the shelf numbers 0 and 1. Nothing crashes: 0, 1, 10, and 20 are all valid integers. The answers have the right machine type but the wrong meaning. Figure 1 shows this exact failure.

The executed RT program has exactly this shape. Primitive positions 0 and 1 are the shelves; application object identifiers 10 and 20 are the item labels. The intersection callback produces attribute slot zero and any-hit consumes it to emit a pair. The defective producer writes the primitive position rather than the application identifier. The slot is still a valid u32; native access validation, GAS, program groups, pipeline, SBT, and launch all remain legal. The evaluated PyOptiX, active typed-OptiX, and OWL routes therefore reach launch and return (100, 0) and (101, 1). RTDL instead compares the promised meaning with the regenerated callback and rejects before launch.

The error is not a bad number; it is a broken agreement about what the number means. We call that agreement a cross-role semantic ABI. Checking it requires relating the producer, consumer, physical binding, and launched executable. The same whole-program view is needed for effects, capacity status, and continuation.

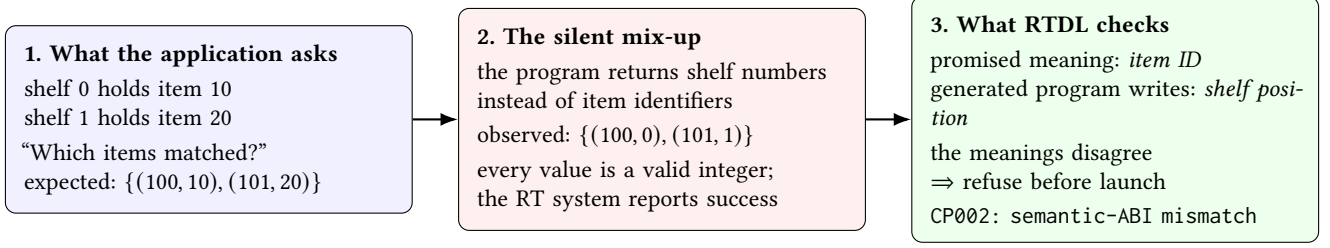
The failure does not depend on spatial expertise. In a database join it is the difference between a row identifier and a storage position; in sorting it is the difference between a persistent record identifier and its current array position. We use the spatial instance because it is the actual executed counterexample. Sorting is intuition only: RTDL neither discovers a sorting mapping nor proves a comparator, rank computation, or scatter correct.

An application test catches this particular input because item IDs 10 and 20 differ from shelf positions 0 and 1. If the numbers happened to coincide, that test could hide the bug. More generally, tests sample inputs; they do not reject the broken agreement itself. RTDL therefore complements rather than replaces testing; the oracle checks observed answers, while admission checks the declared protocol before any answer is produced.

## 3 Whole-Protocol Admission

### 3.1 Compilation unit

The restricted language defines seven roles: bounds, ray construction, intersection, any-hit, closest-hit, miss, and finalization. These are language roles rather than seven OptiX entry points. A trusted ray-generation wrapper invokes the



**Figure 1.** Everyday view of the executed counterexample. In the actual RT program, a shelf number is an OptiX primitive position, an item label is an application item identifier, and the handoff is attribute slot zero. The exact correct and defective rows are preserved by the evaluation. RTDL does not decide which objects match; it checks that the callbacks agree on what the handoff number means. The figure isolates CP002, not all five obligations.

appropriate leaves and owns the trace/finalization boundary. Restricted source is parsed as text and never imported as Python. Verification produces typed Callback IR; subsequent ABI compilation re-verifies that IR and derives deterministic role inputs, effect variants, symbols, and status codes. Resource and geometry obligations remain in the verified IR and physical authority, while continuation is projected from status and family-specific contracts.

The public GPU interface is narrower than that frontend. A caller selects one of two protocol specifications, supplies its parameters, the sole standard physical plan, an external proof identity, and family-specific inputs. The compiler then re-derives the exact compiler-owned source and manifest for that family. Caller-supplied restricted source and Callback IR are not GPU extension points. The standalone restricted-source path supports verification and CPU interpretation only.

### 3.2 Five admission obligations

**Cross-role effects.** Each role declares permitted reads and writes and, where required, state that must be produced for downstream roles. The verifier rejects a declaration that omits a required effect or grants an effect outside the admitted topology.

**Semantic ABI ownership.** The ABI records payload and attribute metadata. The evaluated CP002 leaf is the nominal meaning of attribute slot zero; payload nominal ownership is not demonstrated. Matching bit widths are insufficient: the attribute producer and consumer must agree on its declared meaning and owner.

**Physical binding.** Callback assumptions are checked against the typed custom-AABB or built-in triangle physical plan, including field-to-geometry bindings. This prevents a coherently typed but semantically permuted geometry layout.

**Status-gated continuation and completeness.** Device status and capacity are part of the protocol. Host continuation must observe status, and a failed or truncated execution

cannot expose an application result. Merely reading status before copying output is insufficient when a non-OK capacity condition does not suppress that output.

**Checked executable identity.** In the current materializer, the two CP005 fields carry the same selected executable digest. Their comparison is identity-binding instrumentation, not independent extraction. The later content-addressed deployment and execution checks rehash source/PTX, proof and physical authorities, target, provider, and native identities to establish the executable chain of custody.

These categories are related rather than five independent theorems. Effect closure and status are hierarchical, while semantic ABI and physical binding form a shared schema layer. Executable identity carries the other decisions to the launched bytes and is treated as identity-binding support, not as a fifth independent novelty count.

### 3.3 Public lifecycle and scope

The stable public GPU surface exposes exactly two compiler-owned closed families:

1. a custom-AABB bounded relation that emits canonical pairs with an explicit capacity; and
2. a built-in-triangle all-hit computation followed by a checked unsigned 64-bit reducer.

For either family, materialize validates the protocol, re-derives its compiler-owned callback program, compares its internal projections, and binds the result to a target without creating GPU state. prepare creates a process-local, thread-affine owner; execute admits a dynamic batch and returns output only after status and identity checks; and close destroys the owner. Raw source/IR, PTX, entry points, program groups, SBT records, pipelines, and native handles are not user inputs.

The implementation supports trace depth one and callable depth zero. The public surface excludes arbitrary recursion, dynamic dispatch, raw pointers, arbitrary atomics, nested optixTrace, user callables, curves, spheres, motion, instancing, micromaps, arbitrary multi-report intersections, new geometry families, and new role or continuation topologies.

These are capability limits, not future features implicitly credited to the current system.

### 3.4 Admission judgment

During materialization, the implementation constructs two internal projections. The declaration-side projection  $D = \langle f, t, E, A, B, C, X \rangle$  contains protocol family  $f$ , task-semantics identity  $t$ , IR-side role/effect map  $E$ , evaluated attribute-slot ownership map  $A$ , visible physical binding map  $B$ , continuation policy  $C$ , and the executable identity  $X$  selected during materialization. The target-side projection  $P$  contains corresponding primed fields plus generated device source  $S_d$  and host materializer  $S_h$ . Its CP001–CP004 facts are separately re-derived from the regenerated callback ABI, physical authority, and compiled contract. For CP005, both projections receive the same materializer-selected executable digest. Maps are sorted and all structures use canonical JSON serialization before hashing. Neither projection is an application-authored certificate.

The admission relation is deliberately simple and auditable:

$$\text{admit}(D, P) \iff (f, t, B) = (f', t', B') \wedge E = E' \wedge A = A' \wedge C = C' \wedge X = X'. \quad (1)$$

Malformed or incorrectly sealed inputs fail before this relation. A failed conjunct emits a stable reason identifier (CP001–CP005) and no executable capability. Family and task identity are included in the physical-binding conjunct: changing the semantic domain cannot masquerade as a sixth generic mechanism.

Figure 2 shows the important boundary. The semantic and physical facts in  $D$  are derived from the verified family specification, compiler-owned IR-side effects, and visible physical plan. Their counterparts in  $P$  come from the regenerated target ABI and compiler-owned physical objects, not by serializing  $D$  again. For CP001–CP004 these are distinct derivation paths inside one implementation, not independent implementations. Their agreement establishes consistency among represented facts under the trusted computing base; it does not prove target-projection correctness or that the application computes the right function. CP005 leaf mutation establishes comparator sensitivity only; the later content-addressed and runtime checks provide its chain-of-custody evidence.

Acceptance records the identities of the program, target, physical schema, contract, ABI, executable, PTX, and native library. Execution rechecks the PTX and native-library identities, validates device status before returning output, verifies the output digest, and binds a traversal receipt to the expected route. Executable identity is therefore a chain from admission to observation; Eq. 1 alone is not a functional correctness proof.

### 3.5 What the worked mismatch establishes

The rows in Figure 1 come from the preserved executions rather than a paper-only illustration. The public RTDL lifecycle returns  $\{(100, 10), (101, 20)\}$  through real OptiX traversal. The corresponding defective callback is legal OptiX, and the evaluated PyOptiX, active typed-OptiX, and OWL routes all reach launch; the fault returns primitive-index rows  $\{(100, 0), (101, 1)\}$ . Replacing only the nominal contract leaf from application-item-ID to primitive-index preserves the machine representation and target projection bytes, yet RTDL changes ACCEPT to the sole CP002 rejection because  $A \neq A'$ .

This establishes a narrower but useful fact: nominal protocol admission can make this declared mismatch impossible to launch even though conventional machine typing and composition accept it. It does not prove that the overlap mapping is the right algorithm, that the declared identifier is truthful, or that an accepted result matches an application oracle. Those remain application obligations.

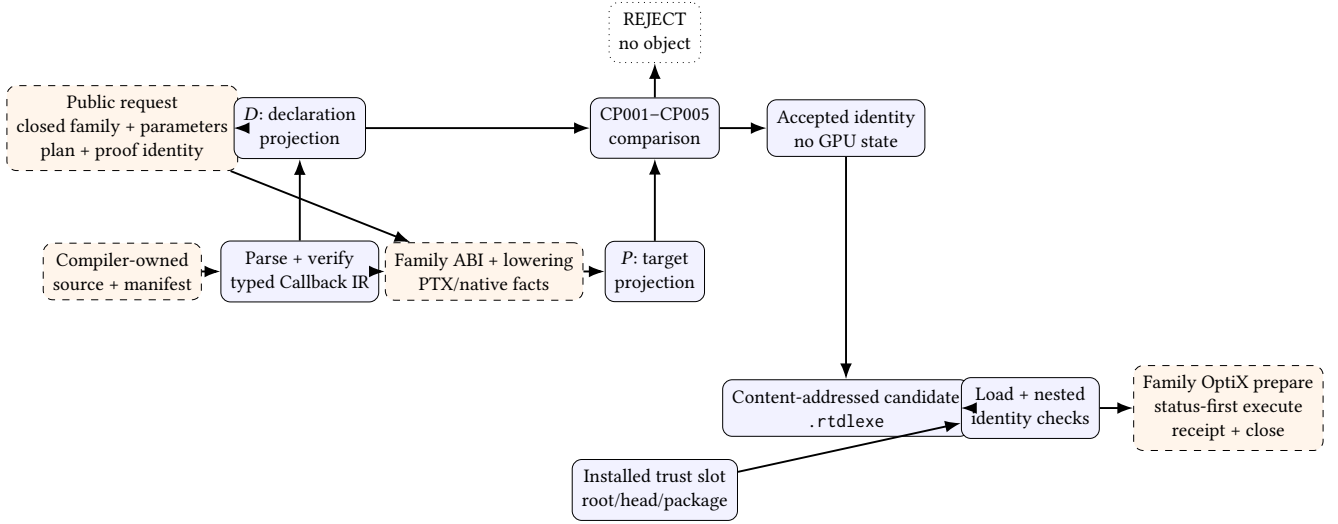
## 4 Implementation and Application Cohort

The implementation contains the base callback path and six M1–M6 application-neutral composition batches: checked triangle reductions, bounded relation emission, prepared multi-round spatial composition, exact predicate/global witness composition, grouped event reduction, and bounded hierarchy-frontier continuation. Selection depends on typed semantics, resource/proof inputs, and physical identity rather than application names.

**Architecture and trust boundary.** The parser, typed Callback IR, canonical ABI, role-leaf generator, PTX composer, projection schemas, exact CP001–CP005 comparator, content addressing, and lifecycle state discipline are reused by the two implemented families. Their compiler-owned source, proof policy, semantic/physical projectors, wrapper and entry symbols, target compiler, native program bundle, GAS/SBT layout, launch path, status reduction, and input/output schemas are family-specific. “Shared” therefore means reuse across these two families, not parametricity or soundness for an unimplemented family.

An accepted materialized object can be frozen as a content-addressed `.rtdlexe`. The artifact carries the checked projections and exact composed PTX but grants no deployment authority by itself. An installed trust root, current signed head, and signed package authorize one exact slot. On a cache hit the loader rechecks protocol, target, provider, native-producer, and deployment identities without importing the frontend, code generator, Numba, or NVRTC; prepare still constructs process-local OptiX objects. The trust layer does not resist an administrator rolling both installed trust files back to an older valid signed pair.

The trusted computing base includes the parser and IR verifier, closed-family providers and physical projectors, ABI



**Figure 2.** Implemented architecture. Solid boxes are mechanisms reused by the two current families; dashed boxes are family-specific. The public GPU interface does not accept arbitrary Callback IR. CP001–CP004 facts are separately re-derived inside one trusted implementation; CP005 has common origin and is backed by downstream identity checks.

compiler, code generator and composer, both projection producers and their comparator, deployment builder/loader and administrator-installed trust state, exact native producer, CUDA/OptiX driver, and GPU. The Python interpreter and standard library, OS filesystem and dynamic loader, process isolation, signing-key custody, deployment administrator, and Python code already controlling the loader process are also trusted. External proof objects are checked for form and exact program binding; their truth is an assumption, and RTDL does not prove the theorem denoted by an opaque digest.

Adding a third protocol family is currently a shared-core and native-backend change, not a user-level registration. It requires a protocol type and standard plan, compiler-owned source and proof policy, both projectors, target compiler and wrapper, executable record, lifecycle dispatch, deployment and runtime projections, native OptiX construction, and tests. Existing applications may reuse either implemented family without application-name dispatch, but that is not evidence that a new callback topology can be authorized without modifying RTDL.

Nine published non-rendering RT systems yield thirteen project-authored lanes. Particle Tracking and RayDB contribute one each [43, 49]. The graph study and LibRTS contribute two each [10, 51]. RTNN, RT-DBSCAN, and X-HD contribute one each [11, 31, 56]; RayJoin contributes three [9], and RT-BarnesHut one [30]. All thirteen ports were written by the RTDL authors and are exact on their frozen inputs while exercising OptiX traversal. They are neither third-party implementations nor claims of complete reproduction of the source systems or their evaluations. This is reuse evidence for two closed families, not evidence that the public

language admits thirteen arbitrary protocols or that a new application will transfer.

## 5 Evaluation

We ask five questions: (1) are populated declaration leaves decision-bearing; (2) do OptiX’s typed-payload semantics or OWL already prevent the same faults; (3) what capability is actually executable; (4) can an application outside the original fixture reuse a frozen family without a core change; and (5) what lifecycle-separated cost remains relative to PyOptiX-compat?

### 5.1 Experimental discipline

Mechanism experiments use two matched non-rendering tasks, nearby legal controls, single-mechanism mutations, frozen expected outputs, and exact oracles. The performance experiment uses identical task semantics, input, precision, output materialization, status-before-output ordering, launch and synchronization meaning, and paired-block statistics. Ratios are RTDL/PyOptiX-compat, so lower is better. Steady execution has a registered 1.05 noninferiority margin; preparation and post-import deployment cold use 1.10. Preparation is reported separately and is never treated as free. A gated row passes only when both its point estimate and the upper endpoint of its 95% interval do not exceed the margin. Within every task/regime, the 18 blocks contain each of the six arm orders three times. Worker medians are integer nanoseconds; half-ns midpoints from even steady samples are truncated toward zero before ratios.

The reported 18-block study is a registered successor experiment, not the first outcome-blind performance test. Two relevant predecessor results were known before its freeze:

**Table 1.** Implementation reuse boundary. “Shared” is an observed property of the two current families, not a general soundness result.

| Layer              | Shared mechanisms                                                          | Family-specific policy or code                                                                       |
|--------------------|----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Frontend/admission | AST-only parser, typed IR, canonical ABI and identities, exact CP001–CP005 | source/manifest, role topology, proof policy, semantic labels, physical bindings and both projectors |
| Lowering           | verified role-leaf generation and deterministic composition                | physical schema, wrapper, entry symbols, compiler options and target compiler                        |
| Deployment         | content addressing, signed slot selection and nested identity verification | provider/native bundle, SBT layout, launch and I/O schemas                                           |
| Execution          | lifecycle states plus identity/status/output checks                        | GAS/program groups/SBT construction, launch, status reduction, output and receipt route              |

**Table 2.** Exact scope of the paper-derived cohort.

| Published system    | Selected RTDL lane(s)                         | Family   | Not claimed                             |
|---------------------|-----------------------------------------------|----------|-----------------------------------------|
| Particle tracking   | cell_transition                               | relation | full simulation                         |
| RayDB               | q21                                           | relation | full query suite or source reproduction |
| RT graph case study | rt_1a2, rt_2a1                                | triangle | other graph algorithms                  |
| LibRTS              | overlap_filter, range_rows                    | relation | dynamic indexing/instancing             |
| RTNN                | ranked_window                                 | relation | full scheduling system                  |
| RT-DBSCAN           | components                                    | relation | full preprocessing/border policy        |
| X-HD                | global_witness                                | relation | pruning/load balancing                  |
| RayJoin             | segment_pairs, point_location, grouped_events | relation | complete system/evaluation              |
| RT-BarnesHut        | force                                         | relation | tree construction                       |

**Table 3.** Capability levels must not be conflated.

| Level                  | Demonstrated scope                         | Count                |
|------------------------|--------------------------------------------|----------------------|
| Published-system ports | exact project-authored selected lanes      | 9 systems / 13 lanes |
| Composition layer      | base path plus application-neutral batches | 6 batches            |
| Stable public GPU API  | closed executable protocol families        | 2 families           |
| Prospective exam       | unbiased new-application transfer          | 0                    |
| External author        | third-party human application authoring    | 0                    |

a 24-block three-arm RTX A4500 study whose six RTDL-versus-PyOptiX-compat gates all failed, and a 16-block two-arm RTX 4000 Ada study whose two steady gates passed and four setup gates failed. An earlier asymmetric benchmark was also withdrawn after the baseline’s avoidable Python work exceeded its apparent advantage; none of its timings or claims is reused. The successor fixed its arms, tasks, regimes, thresholds, complete schedule, estimator, and unconditional acceptance rule before its own timings. It reran all three arms in 324 fresh processes, reused no predecessor worker or timing, and accepted every outcome unconditionally.

## 5.2 Every populated contract leaf affects admission

The two accepted task contracts contain 19 populated semantic leaves. We mutate each leaf alone while holding compiler projection bytes unchanged. All 19 mutations change ACCEPT to REJECT; each produces exactly one expected finding, and no leaf is classified as non-decision-bearing. This includes the safety-critical finalize: require\_status\_ok leaf. A standard-library verifier that imports neither RTDL nor Py-OptiX checks and recounts the preserved 19-row projection, including the recorded mutations, decisions, and reason identifiers.

This result establishes liveness only for populated leaves in these two contracts. It does not establish completeness of the obligation vocabulary or semantic correctness of arbitrary accepted applications.

## 5.3 Residual under active typed OptiX and OWL

OptiX 9 exposes an important native mechanism that must receive credit: OptixPayloadType assigns read/write semantics per 32-bit payload word and shader stage [33]. In a separate strongest-native configuration, we enabled that mechanism, selected the type at both trace sites and every program group, and enabled validation ALL. A source/PTX-identical negative that removed any-hit rights was rejected during module creation, establishing that the mechanism was active. Nevertheless, the preregistered typed-route realizations of all five counterexample mechanisms built, linked, launched, emitted no validation error or fatal message, and reproduced their registered consequences. Thus the result is not an experiment against an omitted native facility.

The CP002 path is particularly sharp: intersection reports either an application item identifier or a primitive index in

**Table 4.** Experiment-to-claim map. Each row states both the supported claim and the inference that the experiment cannot support.

| Question                  | Falsifier                                                                                 | Observation                                                                                                                             | Claim ceiling                                                                                                                                      |
|---------------------------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Contract liveness         | a populated leaf changes without changing admission                                       | 19/19 single-leaf mutations change ACCEPT to REJECT; projection bytes fixed                                                             | populated leaves in two contracts are decision-bearing; vocabulary completeness is unproven                                                        |
| Residual over mature APIs | typed OptiX or OWL rejects or diagnoses its preregistered realization before launch       | typed-route realizations of five mechanisms launch under both evaluated routes; a typed-payload negative rejects; RTDL rejects all five | three full wrong-output residuals, one partial status-gated continuation/completeness residual, and binding support; not five independent theorems |
| Executable scope          | public lifecycle executes fewer than its declared families or authored routes are inexact | two public families execute; 13/13 authored lanes are exact and behavioral OptiX                                                        | authored diversity, not arbitrary Callback IR or prospective generalization                                                                        |
| Existing-family reuse     | selected cross-domain task needs another family/core change                               | SQL bag equijoin reuses bounded relation without another core change; BED failure preserved                                             | one constructive project-selected reuse, not blind or unbiased transfer                                                                            |
| Lifecycle cost            | matched RTDL exceeds registered margins                                                   | steady passes 1.05 on two tasks; cold and prepare fail 1.10; same-target Direct rows reported                                           | two tasks/one target; Direct rows descriptive; no OWL timing claim                                                                                 |

attribute slot zero; any-hit reads that slot and writes the relation row directly. No payload access occurs on this defective path. The examined OptiX headers declare the number of attribute values and numbered attribute accessors, but expose no per-slot nominal-meaning declaration. This is a scoped observation about the inspected OptiX 9 headers, not a claim about every NVIDIA interface.

A pinned NVIDIA OWL arm with validation ALL enabled by a diagnostic-only overlay separately receives full credit for building acceleration structures, programs, pipeline, SBT, and launch, and executes its five preregistered invalid protocol realizations. The overlay is the sole upstream modification and changes none of those objects, the launch, or the output. This is functional GTX1070/Pascal evidence, not RT-core or performance evidence. The examined public API at the pinned OWL revision exposes payload and attribute counts but not the native payload-type declaration; obtaining that facility requires leaving the examined abstraction or modifying its implementation. Table 5 reports the OWL result per mechanism rather than treating five categories as five independent novelties.

An exact application oracle detects all five faults on the registered exposing inputs. The active typed route emits no validation error or fatal message for any of the five. The OWL overlay, which records only error/fatal levels, also records none; lower-severity OWL diagnostics were not observed and are not claimed absent. The contribution is therefore pre-launch, input-coverage-independent rejection of declared inconsistencies, not removal of the need for an oracle. The strongest-native execution covers two project-designed tasks in one OptiX 9 environment with a GTX 1070 and required a disclosed two-line FFI repair to transmit native payload-type

declarations; it is neither stock-PyOptiX nor modern-RTX evidence.

#### 5.4 One constructive reuse and one preserved failure

A SQLite integer bag equijoin over a bounded non-null integer domain maps each key  $k$  to the exactly representable binary32 AABB  $[k, 0, k + 1, 1]$ . Equal keys overlap with area one and unequal integer keys with area at most zero; row identifiers preserve duplicate Cartesian multiplicity. The application reuses the frozen bounded-relation family, matches independent SQLite and Python oracles, and requires no further core, native, geometry-family, or protocol-family change.

For a database reader, the CP002 distinction is exactly “row identifier versus physical AABB position.” We did not rerun the five invalid-control executions on the SQL input, so this is evidence of existing-family semantic reuse, not a second residual experiment.

This task was selected by the project after a BED interval-intersection transfer exposed a missing public overflow mapping and required a core repair. The SQL result is consequently a constructive, post-failure, existing-family reuse case, not a blind or prospective exam. We preserve both outcomes and report a prospective unbiased new-application exam count of zero.

#### 5.5 Steady noninferiority passes; setup does not

The baseline denoted PyOptiX-compat is a pinned NVIDIA otk-pyoptix 9.1.0 source build using the OptiX 9.0 compatibility API. It is not a stock/current OptiX-9.1-API arm. The same-target study uses current RTDL and Direct source plus this pinned compatibility arm.

**Table 5.** Executed residual after crediting OWL for host composition. All nearby valid controls are exact and every corresponding RTDL mutation is rejected before launch. A separate active OptiX 9 typed-payload configuration also launches preregistered realizations of all five mechanisms.

| Obligation                             | OWL reached launch | Frozen consequence                                   | RTDL decision     | Claim strength               |
|----------------------------------------|--------------------|------------------------------------------------------|-------------------|------------------------------|
| Role/effect closure                    | yes                | wrong weighted result                                | pre-launch reject | full executed residual       |
| Evaluated attribute-slot semantics     | yes                | wrong relation pairs                                 | pre-launch reject | full executed residual       |
| Physical geometry binding              | yes                | swapped relation pairs                               | pre-launch reject | full executed residual       |
| Status-gated continuation/completeness | yes                | overflow observed before copy, yet 7/8 rows consumed | pre-launch reject | partial enforcement residual |
| Checked executable identity            | yes                | different executable result                          | pre-launch reject | binding support              |

This is the registered successor described above; “registered” here does not mean outcome-blind to the predecessor results. It contains 324 fresh workers and 7,128 registered timing observations across matched Direct CUDA/OptiX, PyOptiX-compatible, and RTDL arms. An independently implemented raw recount reconstructs the schedule from 324 raw receipts and reproduces every worker median and all 18 reported comparison rows, each computed from 18 paired-block ratios, including every confidence interval. Table 6 shows all three comparisons. Only RTDL/PyOptiX-compatible carries a registered noninferiority gate; Direct comparisons are descriptive.

The result supports a lifecycle-specific statement. On these two tasks, the steady median arm differences are  $-17\ \mu\text{s}$  and approximately  $+1.22\ \mu\text{s}$ ; both pass the registered 1.05 noninferiority gate. This is not evidence of equality or zero execution cost. The observed cold-deployment (after imports) and prepare setup differences are instead 162–223 ms. Direct is 1.36–2.83 $\times$  faster than both Python-host arms at steady state. Conversely, the compatibility arm prepares faster than both RTDL and the matched Direct implementation. The experiment measures complete arm boundaries and has no checker-off RTDL ablation; it therefore cannot allocate the setup difference to checking, identity, compilation, loading, or another component. The defensible result is lifecycle-specific arm cost, not a causal micro-cost attribution.

Each reported ratio is the median of 18 paired block ratios, so ratios from different columns are not algebraically composable. Confidence intervals use 10,000 fixed-seed resamples of the 18 block ratios via repeated `randrange(18)` draws; the sorted draws at indices 249 and 9749 are the reported bounds. Cold and prepare therefore have 18 arm observations per row, whereas each steady worker first summarizes 64 registered executions.

The surprising Direct setup row is not explained by compilation. Frozen source shows that all three arms consume pre-built device artifacts: the Direct and compatibility arms use

the same matched PTX, while RTDL uses family-specific embedded PTX. Direct invokes no NVRTC on the formal route. Each arm constructs its context/module, program groups, pipeline, SBT, task owner, and static allocations or GAS inside its prepare boundary, although the implementation paths are not byte-identical. Only Direct records matching subphase timers. Consequently, Direct’s 459–472 ms setup medians and their difference from the compatibility arm remain descriptive; the current evidence does not identify their cause.

## 6 Programming Responsibility, Not Usability

For the two admitted families, RTDL owns validation and generation of the cross-role contract, physical-plan binding, executable identity, lifecycle, and status-gated output boundary. The user does not supply raw PTX, an SBT, a pipeline, or a native loader to the public lifecycle. The executed faults show that these are real responsibilities rather than a line-count rubric.

No third party has authored an RTDL application. We have no human usability study, developer-time measurement, or functionally matched productivity baseline. We therefore make no claim that RTDL is easier, shorter, or more productive than CUDA/OptiX, PyOptiX, or OWL. The measured contribution is responsibility transfer and pre-launch rejection of declared inconsistencies. Learnability and authoring efficiency remain open empirical questions.

## 7 Related Work

Table 8 organizes prior work by primary system responsibility rather than implementation form or surface syntax. Systems can span classes. Repurposed applications are the workload population to which these responsibilities apply, not a fifth programming-system class.

**Coverage of the taxonomy.** The four responsibilities are a problem-scoped organization, not a claim that every RT

**Table 6.** Same-target three-arm ratios on RTX 4000 Ada: current RTDL/Direct source and pinned PyOptiX-compat. Every cell is numerator/denominator with a 95% CI; lower favors the numerator. Only RTDL/PyOptiX-compat has a gate.

| Task     | Regime  | RTDL/PyOptiX-compat | PyOptiX-compat/Direct | RTDL/Direct         | Gate      |
|----------|---------|---------------------|-----------------------|---------------------|-----------|
| Relation | cold    | 1.777 [1.735,1.862] | 0.583 [0.553,0.624]   | 1.083 [0.993,1.122] | fail 1.10 |
| Relation | prepare | 1.788 [1.720,1.951] | 0.597 [0.566,0.606]   | 1.082 [0.997,1.142] | fail 1.10 |
| Relation | steady  | 0.973 [0.967,0.982] | 2.830 [2.801,2.850]   | 2.753 [2.730,2.766] | pass 1.05 |
| Triangle | cold    | 1.488 [1.402,1.544] | 0.731 [0.666,0.770]   | 1.061 [1.019,1.103] | fail 1.10 |
| Triangle | prepare | 1.477 [1.387,1.564] | 0.723 [0.676,0.754]   | 1.057 [1.005,1.102] | fail 1.10 |
| Triangle | steady  | 1.018 [1.013,1.026] | 1.360 [1.355,1.368]   | 1.387 [1.383,1.393] | pass 1.05 |

**Table 7.** Absolute per-arm medians from the same experiment. Cold and prepare workers contribute one observation each; steady workers contribute 64. The final column is an observed RTDL–PyOptiX-compat arm difference, not a causal allocation to checking or admission. Deltas use unrounded medians, so displayed arm values need not subtract exactly.

| Task     | Regime  | Direct  | PyOptiX-compat<br>milliseconds | RTDL    | $\Delta$ |
|----------|---------|---------|--------------------------------|---------|----------|
| Relation | cold    | 472.149 | 275.878                        | 498.380 | +222.5   |
| Relation | prepare | 459.175 | 273.205                        | 492.011 | +218.8   |
| Relation | steady  | 0.259   | 0.729                          | 0.712   | −0.017   |
| Triangle | cold    | 465.241 | 341.094                        | 517.583 | +176.5   |
| Triangle | prepare | 464.681 | 340.006                        | 502.438 | +162.4   |
| Triangle | steady  | 0.050   | 0.067                          | 0.069   | +0.001   |

tool is a direct competitor. HLSL/DXR and SPIR-V/Vulkan remain in the interface class. Renderer-domain languages such as OSL and MDL define shading or material computation inside renderer-owned protocols [1, 34], placing them primarily in interface/construction rather than repurposed mapping or protocol admission. Embree and HIP RT expose ray-traversal kernels or libraries [2, 40] and likewise fall under interface/construction. General parallel DSLs such as Taichi are mapping/generation substrates when used to emit accelerator code [48]. These systems are not ignored; they are less direct comparisons because they do not document the bounded joint admission relation evaluated here. We make no fault-survival claim for systems we did not execute.

**Interface systems.** OptiX supplies a programmable ray-tracing engine [37]. Its payload types express per-word, per-stage access rights [33]; DXR Payload Access Qualifiers provide per-field access contracts and require each declared read stage to follow a write and each declared write stage to precede a later read [27]; and Vulkan defines matching ray-payload and hit-attribute shader interfaces [19]. These mechanisms are closer prior art than an untyped raw-OptiX baseline and are credited explicitly. They constrain access and, for DXR payloads, part of producer/consumer stage ordering. CP001’s exact executed mutation changes callback control semantics from `optixIgnoreIntersection` to `optixTerminateRay`; it is not a missing payload-stage declaration. Even when an interface fixes machine representation and access, it need not express nominal application meaning. CP002 sharpens this residual for attribute slot zero: the

inspected OptiX interface exposes the attribute count and numbered accessors, but no per-slot nominal declaration. The evaluated RTDL residual otherwise concerns nominal attribute meaning and whole host/device protocol obligations that survive the active OptiX mechanism. We ran no DXR executable arm and make no cross-API survival claim. OptiX Utility spans interface and construction through typed payload/attribute signatures, automatic packing, usage annotations, and host-side object helpers [44]. We credit these structural and packing mechanisms; we did not execute that library and make no claim about faults it would catch beyond its documented mechanisms. Slang’s capability system likewise validates target, API, stage, and feature requirements [45]. Exposing OptiX payload types through another construction layer would strengthen access checking, but it would not by itself supply nominal application intent.

**Construction systems and the OWL boundary.** PyOptiX is a broad Python host binding [35] and can implement the same applications manually. Shader Components introduced first-class modules that encapsulate shader logic with their parameters while retaining static specialization and efficient binding [14]. Slang showed that constrained generics, interfaces, associated types, parameter blocks, and reflection can implement modular and extensible shading systems without a proprietary shader DSL [13]. Slang could compile individual RTDL leaves, and RTDL claims none of these language, specialization, or parameter-binding mechanisms.

OWL makes an essential step beyond raw OptiX: it lifts acceleration structures, programs, pipelines, SBT construction, resources, and launch into a managed composition layer [36]. We grant it full credit for that responsibility. At the pinned revision, however, OWL’s public contract carries structural composition facts while the application supplies device programs and selects callback entries; it requires no explicit application-semantic authority for nominal slot meaning, required cross-role effects, semantic geometry projection, status-gated consumption, or an admitted identity carried to the selected executable. Successful composition therefore establishes that an RT object graph is legal, not that it implements the intended application protocol.

OWL variable names and types can document local intent and enforce host-record layout, but OWL does not require those declarations to form a cross-role application-semantic

**Table 8.** Four RT programming-system responsibilities. Placement records the primary responsibility relevant here; it does not deny secondary capabilities.

| Class                                       | Primary question                                                                                 | Representative systems                                                              | Boundary relative to RTDL                                                                                                       |
|---------------------------------------------|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Interface systems                           | Are shader stages, declared interfaces, accesses, and target capabilities legal?                 | OptiX typed payloads; DXR PAQ; Vulkan interfaces; Slang capabilities; OptiX Utility | Constrain declared structure, access, sequencing, and target legality; do not jointly relate the complete application protocol. |
| Construction systems                        | How are modules, acceleration structures, pipelines, SBTs, resources, and launches assembled?    | PyOptiX; OWL; Slang and Shader Components                                           | A legal object or module graph can still implement the wrong application protocol.                                              |
| Mapping systems                             | How is a high-level computation generated, specialized, or translated for an RT-capable backend? | Dr.Jit; LuisaRender and LuisaCompute; CrossRT                                       | May enforce generated-code invariants, but do not document the joint application-semantic admission relation studied here.      |
| Proposed responsibility: protocol admission | Is the assembled executable admitted against one declared application protocol before launch?    | RTDL                                                                                | Bounded here to two closed OptiX families; not a general protocol type system or arbitrary Callback-IR backend.                 |

authority. This separation does not depend on the pinned revision’s public omission of `OptixPayloadType`. Exposing native payload access declarations would strengthen interface checking, but would not supply the nominal authority, physical obligations, continuation policy, or executable-selection contract considered here.

More fundamentally, the missing information cannot be recovered from composition alone. Fix an executable and a legal OWL object graph. That same pair can be correct for an application whose attribute-zero contract denotes a primitive index, yet incorrect for one whose contract denotes an application item identifier. Without a separately supplied or otherwise formalized intended protocol, no checker confined to the executable and OWL’s structural contract can know which interpretation is required. This is not a missing validation switch: resolving it requires adding application-semantic authority to the abstraction. RTDL does not infer which intent is correct; it requires the application to state that authority and checks a bounded implementation against it.

In our pinned executable route, each invalid realization and its nearby legal control use the same OWL composition skeleton and slot layout for that task; all build and launch, whereas RTDL rejects each corresponding declared mismatch before launch (Table 5). RTDL makes the additional authority explicit only for two closed families and checks it against re-derived implementation facts. OWL could adopt an equivalent abstraction, but doing so would extend its responsibility from object construction to whole-protocol admission.

**Mapping systems.** Dr.Jit traces and globally specializes rendering and differentiable computation [18]; LuisaRender and LuisaCompute provide a cross-platform embedded kernel DSL, JIT compilation, and unified resource runtime [55]; and CrossRT maps portable C++ algorithms to RT backends and software fallbacks, including megakernel and wavefront modes [7]. These systems address construction, specialization, and the upstream algorithm-to-backend mapping problem. RTDL begins after a traversal formulation is selected

and checks the resulting callback protocol. A translator or JIT could emit the additional protocol authority or incorporate an equivalent admission pass; this paper implements neither integration and claims no comparable optimization, differentiation, or cross-platform translation mechanism.

**Protocol admission and foundations.** RTDL is conceptually adjacent to *typestate*, which restricts operations by lifecycle state [46], and to interface automata and multiparty session types, which model temporal component interfaces and global interaction protocols [6, 17]. Static FFI checking and linking types address type safety and reasoning across language boundaries [8, 38], while GPUVerify verifies intra-kernel data races and barrier divergence [3]. RTDL does not contribute a general protocol type theory, progress or type-safety theorem, verified FFI, or GPU-kernel verifier. It instantiates a bounded admission relation over concrete host/device callback artifacts for two closed protocol families and empirically tests whether each populated obligation leaf affects the decision.

**Repurposed-RT application population.** Repurposed-RT applications are the workload population to which these responsibilities apply, not another class of programming system. Individual works may use interface or construction systems, use a mapping system, or themselves contribute the algorithm-to-RT mapping. The recent review’s 59-document bibliometric corpus includes 35 systematically analyzed proposed solutions spanning 32 problems [26]. It characterizes a mapping as a coordinated design of geometric data representation, query rays, and hit-side action. After that mapping is chosen, the surrounding application must establish its correspondence with callbacks, payloads and attributes, host bindings, status handling, and the launched executable unless an equivalent protocol layer is supplied. The nine systems underlying our thirteen author-written lanes demonstrate the breadth of this protocol-bearing setting, not prevalence: we have not established how often published artifacts mechanically enforce these obligations or contain faults. Our narrower result is that selected correspondence obligations

can be moved into pre-launch admission for two closed families; the mapping theorem and semantic oracle remain application responsibilities. Table 9 makes the exact frozen survey-table projection inspectable rather than relying on selected examples alone.

## 8 Artifact and Reproducibility

We constructed a compact anonymous evidence projection for the paper’s five claim layers. It contains all 19 populated-leaf mutation records; the five executed observations for preserved PyOptiX and OWL residual routes, plus the corresponding RTDL decisions; the five strongest-native typed-payload observations and a source/PTX-identical negative control; the capability and transfer ceilings; SQL and separate Python oracle rows; and all registered scalar timing observations from 324 same-target three-arm workers, including the declared compatibility-arm distribution/version/API identity and the 24/16-to-18-block registration lineage plus the withdrawn asymmetric result. Identifying development history, machine paths, process identifiers, endpoints, and review records are excluded by construction.

A standard-library-only verifier rehashes every manifested payload, checks and recounts the preserved 19-row liveness projection and the 3 + 1 + 1 PyOptiX/OWL residual classification, checks the preserved five-row strongest-native projection and live negative control against frozen values, checks the preserved SQL/Python equality projection, and recomputes all 18 reported comparison rows, each from 18 paired-block ratios, including the fixed-seed bootstrap intervals and registered timing count. The performance recount starts from 7,128 scalar observations rather than from Table 6. This supports numerical and claim-boundary review without access to our internal history.

The compact projection is not itself a GPU re-execution package: its mutation and residual records are preserved execution observations, whereas the verifier independently recomputes their stated counts and invariants. A full GPU re-run additionally requires an OptiX SDK, compatible NVIDIA driver and GPU, the pinned residual-route sources, the source for the compatibility arm, and the executable build inputs. We do not equate a successful offline recount with independent reproduction of the hardware executions.

## 9 Threats to Validity and Limitations

The three most consequential evidence gaps are ordered rather than presented as a defensive checklist. First, arbitrary verified Callback IR cannot be authorized for GPU execution: the admission model is demonstrated through two closed protocol families. The falsifying next test is a new protocol topology instantiated after a generic core freeze. Second, the five executed faults are constructed, so their prevalence in published third-party artifacts is unknown; the next test is a preselected, complete-denominator field

corpus. Third, every application was authored by the project, so external authoring and learnability are unknown; the next test is a frozen-interface external-author case series. These gaps respectively limit mechanism generality, real-world importance, and usability evidence.

**Generality.** There are exactly two closed public GPU protocol families. The thirteen-lane cohort is project-authored. A third topology currently requires shared-core and native-backend changes. The SQL reuse case was selected after a failed transfer. Prospective unbiased exams and third-party authors are both zero. These facts limit external validity and are stated in the abstract rather than deferred to artifact documentation.

**Mechanism scope.** The 19-leaf sweep covers populated leaves in two accepted task contracts. It does not prove that the vocabulary is complete or that every future leaf is decision-bearing. The five categories are related; we do not infer five independent theoretical contributions from five rejection rows.

**Oracles.** Exact task oracles detect all five registered faults on exposing inputs. RTDL does not prove arbitrary output correctness. Its advantage is rejecting the declared mismatch before launch and without relying on which inputs were sampled.

**Performance.** The current matched result uses two authored tasks and one RTX 4000 Ada. The cold regime starts after Python import/runtime preload and is not full-process cold. Preparation fails the registered margin. The baseline is PyOptiX-compatible, not a stock/current OptiX-9.1-API arm. Same-target Direct rows using current source are descriptive and have no performance gate. There is no OWL timing arm, checker-off preparation ablation, second target, or third task.

**Usability.** The public lifecycle is executable but has not been used by an external human author. We make no ease or productivity claim.

## 10 Conclusion

Existing RT systems principally validate interfaces, construct pipelines, or map algorithms to backends. RTDL studies a fourth, bounded responsibility: admitting a repurposed RT program as one declared bounded whole protocol rather than as isolated shaders. It admits declarations, physical plans, executable identity, and lifecycle obligations as one unit. For two accepted contracts, all 19 populated leaves are decision-bearing under single-leaf mutation with projection bytes fixed: 7 role/effect, 1 semantic-ABI, 7 physical-binding, 2 continuation/completeness, and 2 executable-identity leaves. This is contract liveness evidence, not a soundness theorem or proof that the obligation vocabulary is complete.

Under active native OptiX typed-payload semantics exposed through a disclosed two-line repaired PyOptiX FFI, all

**Table 9.** Frozen survey-derived application landscape. The public survey performance table yields 35 paper–problem rows, 32 distinct problem labels, and 29 unique cited works. The four themes are presentation groupings, not a new survey taxonomy; this is neither the survey’s complete 59-document bibliometric corpus nor an enforcement census.

| Application theme                   | Exact frozen problem labels                                                                                                                                                | Source works                                                                                                                                                                                          |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data, indexing, and graph analytics | SpMM; BFS; Triangle Counting; Set Intersection; Binary Search; Point Queries; Range Queries; RMQ; Non-euclidean kNN; ANN; Outlier Detection; Index Scan; kNN; DBSCAN; FRNN | RTSpMM; RT-core graph case study; LibRTS; RTIndex; More Bang for Your Buck(et); RTXRMQ; Arkade; JUNO; RTOD; RTScan; RT-kNNS Unbound; RT-DBSCAN; RTNN [10, 15, 16, 22–25, 31, 32, 50, 51, 53, 56]      |
| Geometry and spatial predicates     | Penetration Depth; Discrete CD; Continuous CD; Line-Segment Intersection; Point in Polygon; Point Location                                                                 | RTPD; hardware-accelerated discrete/continuous collision detection; RayJoin; unstructured-mesh point location [9, 20, 29, 47]                                                                         |
| Physics and simulation              | Barnes-Hut; Particle Simulation; Radio Wave Propagation; Particle Tracking; Particle-Mesh Coupling; Infrared Radiation; Particle Transport                                 | RT-BarnesHut; particle-based simulation; RT radio-wave propagation; particle tracking; particle-mesh coupling; RT infrared radiation; RTSRT; OpenMC particle transport [4, 5, 12, 21, 30, 41, 49, 54] |
| Visualization and representation    | Graph Drawing; Space Skipping; Segmentation; Voxelization                                                                                                                  | RT-core graph drawing; hardware-accelerated space skipping; GPU sparse ray-traced segmentation; surface/solid voxelization [28, 39, 42, 52]                                                           |

five preregistered fault realizations reached launch after a source/PTX-identical negative established that native access checking was active. A separate pinned OWL arm also built and launched all five. The scientific result is deliberately not “five full guarantees”: it is three full wrong-output residuals, one partial status-gated continuation/completeness residual, and checked-executable binding support. RTDL rejected the corresponding declared inconsistencies before launch.

The implementation remains bounded to two closed public families and project-authored ports. Against the pinned otk-pyoptix 9.1.0 compatibility arm described above, steady RTDL/PyOptix-compat passed the registered 1.05 noninferiority gate on two tasks, with median differences of  $-17\ \mu\text{s}$  and  $+1.22\ \mu\text{s}$ ; post-import deployment-cold and prepare instead showed 162–223 ms setup deltas. The Direct arm is descriptive. Prospective transfer, third-party authorship, usability, arbitrary Callback-IR execution, and cross-API survival remain unestablished.

## References

- [1] Academy Software Foundation. 2026. Open Shading Language Documentation. <https://open-shading-language.readthedocs.io/en/latest/>. Programmable shading for advanced renderers.
- [2] AMD GPUOpen. 2026. HIP RT: A Ray Tracing Library for HIP. <https://github.com/GPUOpen-LibrariesAndSDKs/HIPRT>.
- [3] Adam Betts, Nathan Chong, Alastair F. Donaldson, Shaz Qadeer, and Paul Thomson. 2012. GPUVerify: A Verifier for GPU Kernels. In *Proceedings of the ACM International Conference on Object-Oriented Programming Systems Languages and Applications*. 113–132. doi:10.1145/2384616.2384625
- [4] Ka-Hou Chan, W. Ke, and Sio Kei Im. 2018. Particle–Mesh Coupling in the Interaction of Fluid and Deformable Bodies with Screen Space Refraction Rendering. *Computer Animation and Virtual Worlds* 29 (2018).
- [5] Cunhao Cui, Tiejun Li, Jianmin Zhang, Hanqing Li, Changsong Jin, and Ruixuan Ren. 2024. RTSRT: Accelerating Monte Carlo Particle Transport with Ray Tracing Shared Cache Architecture. *2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)* (2024), 600–606.
- [6] Luca de Alfaro and Thomas A. Henzinger. 2001. Interface Automata. In *Proceedings of the 8th European Software Engineering Conference Held Jointly with the 9th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 109–120. doi:10.1145/503209.503226
- [7] Vladimir Frolov, Vadim Sanzharov, Garifullin Albert, Maxim Raenchuk, and Alexei Voloboy. 2024. CrossRT: A Cross Platform Programming Technology for Hardware-Accelerated Ray Tracing in CG and CV Applications. *arXiv preprint arXiv:2409.12617* (2024). doi:10.48550/arXiv.2409.12617
- [8] Michael Furr and Jeffrey S. Foster. 2005. Checking Type Safety of Foreign Function Calls. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*. 62–72. doi:10.1145/1065010.1065019
- [9] Liang Geng, Rubao Lee, and Xiaodong Zhang. 2024. RayJoin: Fast and Precise Spatial Join. In *Proceedings of the 38th ACM International Conference on Supercomputing*. 124–136. doi:10.1145/3650200.3656610
- [10] Liang Geng, Rubao Lee, and Xiaodong Zhang. 2025. LibRTS: A Spatial Indexing Library by Ray Tracing. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 396–411. doi:10.1145/3710848.3710850
- [11] Liang Geng, Zhehu Yuan, Rubao Lee, Fusheng Wang, and Xiaodong Zhang. 2026. X-HD: Fast Hausdorff Distance Computation with Ray Tracing. In *Proceedings of the 40th ACM International Conference on Supercomputing*. 945–959. doi:10.1145/3797905.3800509
- [12] Shinya Hashinoki, Satoshi Ohshima, Takahiro Katagiri, Toru Nagai, and Tetsuya Hoshino. 2023. Implementation of Radio Wave Propagation Using RT Cores and Consideration of Programming Models. *2023 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)* (2023), 673–681.
- [13] Yong He, Kayvon Fatahalian, and Theresa Foley. 2018. Slang: Language Mechanisms for Extensible Real-Time Shading Systems. *ACM Transactions on Graphics* 37, 4, Article 141 (2018), 13 pages. doi:10.1145/3197517.3201380
- [14] Yong He, Theresa Foley, Teguh Hofstee, Haomin Long, and Kayvon Fatahalian. 2017. Shader Components: Modular and High Performance Shader Development. *ACM Transactions on Graphics* 36, 4, Article 100 (2017), 11 pages. doi:10.1145/3072959.3073648
- [15] Justus Henneberg and Felix Martin Schuhknecht. 2023. RTIndex: Exploiting Hardware-Accelerated GPU Ray Tracing for Database Indexing. *Proc. VLDB Endow.* 16 (2023), 4268–4281.
- [16] Justus Henneberg, Felix Martin Schuhknecht, Rosina F. Kharal, and Trevor Brown. 2024. More Bang for Your Buck(et): Fast and Space-Efficient Hardware-Accelerated Coarse-Granular Indexing on GPUs. *2025 IEEE 41st International Conference on Data Engineering (ICDE)* (2024), 1320–1333.
- [17] Kohei Honda, Nobuko Yoshida, and Marco Carbone. 2016. Multiparty Asynchronous Session Types. *J. ACM* 63, 1, Article 9 (2016), 67 pages. doi:10.1145/2827695
- [18] Wenzel Jakob, Sébastien Speierer, Nicolas Roussel, and Delio Vicini. 2022. Dr.Jit: A Just-In-Time Compiler for Differentiable Rendering. *ACM Transactions on Graphics* 41, 4 (2022). doi:10.1145/3528223

- [19] Khronos Group. 2026. Vulkan Specification: Shader Interfaces. <https://docs.vulkan.org/spec/latest/chapters/interfaces.html>.
- [20] YoungWoo Kim, Sungmin Kwon, and Duksu Kim. 2025. RTPD: Penetration Depth Calculation Using Hardware-Accelerated Ray Tracing. *The Visual Computer* 41 (2025), 9885–9899.
- [21] Jiaan Liu, Qitai Eri, Bo Kong, Jifeng Huang, and Yue Zhou. 2025. Ray Tracing Core Accelerated High Performance Implementation for Infrared Radiation Signature Simulation Using Reverse Monte Carlo Method. *International Journal of Thermal Sciences* (2025).
- [22] Zihan Liu, Wentao Ni, Jingwen Leng, Yu Feng, Cong Guo, Quan Chen, Chao Li, Minyi Guo, and Yuhao Zhu. 2023. JUNO: Optimizing High-Dimensional Approximate Nearest Neighbour Search with Sparsity-Aware Algorithm and Ray-Tracing Core Mapping. *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (2023).
- [23] Yangming Lv, Kai Zhang, Ziming Wang, Xiaodong Zhang, Rubao Lee, Zhenying He, Yanan Jing, and Xiaoyang Sean Wang. 2024. RTScan: Efficient Scan with Ray Tracing Cores. *Proc. VLDB Endow.* 17 (2024), 1460–1472.
- [24] Durga Keerthi Mandarapu, Vani Nagarajan, Artem Pelenitsyn, and Milind Kulkarni. 2023. Arkade: k-Nearest Neighbor Search With Non-Euclidean Distances Using GPU Ray Tracing. *Proceedings of the 38th ACM International Conference on Supercomputing* (2023).
- [25] Enzo Meneses, Cristóbal A. Navarro, Héctor Ferrada, and Felipe A. Quezada. 2024. Accelerating Range Minimum Queries with Ray Tracing Cores. *Future Generation Computer Systems* 157 (2024), 98–111. doi:10.1016/j.future.2024.03.040
- [26] Enzo Meneses, Cristóbal A. Navarro, Héctor Ferrada, Konstantin Verichev, and Cristian Salazar-Concha. 2027. Ray Tracing Cores for General-Purpose Computing: A Literature Review. *Future Generation Computer Systems* 186, Article 108739 (2027). Available online 3 August 2026. doi:10.1016/j.future.2026.108739
- [27] Microsoft. 2026. DirectX Raytracing Functional Specification: Payload Access Qualifiers. <https://microsoft.github.io/DirectX-Specs/d3d/Raytracing.html>.
- [28] Nate Morrical, Will Usher, Ingo Wald, and Valerio Pascucci. 2019. Efficient Space Skipping and Adaptive Sampling of Unstructured Volumes Using Hardware Accelerated Ray Tracing. *2019 IEEE Visualization Conference (VIS)* (2019), 256–260.
- [29] Nate Morrical, Ingo Wald, Will Usher, and Valerio Pascucci. 2022. Accelerating Unstructured Mesh Point Location With RT Cores. *IEEE Transactions on Visualization and Computer Graphics* 28 (2022), 2852–2866.
- [30] Vani Nagarajan, Rohan Gangaraju, Kirshanthan Sundararajah, Artem Pelenitsyn, and Milind Kulkarni. 2025. RT-BarnesHut: Accelerating Barnes-Hut Using Ray-Tracing Hardware. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 43–56. doi:10.1145/3710848.3710885
- [31] Vani Nagarajan and Milind Kulkarni. 2023. RT-DBSCAN: Accelerating DBSCAN Using Ray Tracing Hardware. In *2023 IEEE International Parallel and Distributed Processing Symposium*. 963–973. doi:10.1109/IPDPS54959.2023.00100
- [32] Vani Nagarajan, Durga Keerthi Mandarapu, and Milind Kulkarni. 2023. RT-kNNS Unbound: Using RT Cores to Accelerate Unrestricted Neighbor Search. *Proceedings of the 37th International Conference on Supercomputing* (2023).
- [33] NVIDIA. 2025. *NVIDIA OptiX 9.0 Programming Guide*. Payload types and per-stage payload semantics. [https://raytracing-docs.nvidia.com/optix9/guide/optix\\_guide.250130.A4.pdf](https://raytracing-docs.nvidia.com/optix9/guide/optix_guide.250130.A4.pdf)
- [34] NVIDIA. 2026. Material Definition Language Documentation. <https://raytracing-docs.nvidia.com/mdl/index.html>. Language and SDK for physically based materials and lights.
- [35] NVIDIA. 2026. PyOptiX: Complete Python Bindings for the OptiX Host API. <https://github.com/NVIDIA/otk-pyoptix>. Pinned experiment source commit 3144f224c0fd18733925faf3d8fb82c7376b8dcf.
- [36] NVIDIA, Ingo Wald, Stefan Zellmann, and contributors. 2026. OWL: The OptiX Wrappers Library. <https://github.com/NVIDIA/OWL>. Experiment pins upstream commit df7390b16bce5244b7352ca6d3e320f838297072.
- [37] Steven G. Parker, James Bigler, Andreas Dietrich, Heiko Friedrich, Jared Hoberock, David Luebke, David McAllister, Morgan McGuire, Keith Morley, Austin Robison, and Martin Stich. 2010. OptiX: A General Purpose Ray Tracing Engine. *ACM Transactions on Graphics* 29, 4, Article 66 (2010). doi:10.1145/1778765.1778803
- [38] Daniel Patterson and Amal Ahmed. 2017. Linking Types for Multi-Language Software: Have Your Cake and Eat It Too. In *2nd Summit on Advances in Programming Languages (Leibniz International Proceedings in Informatics, Vol. 71)*. 12:1–12:15. doi:10.4230/LIPIcs.SNAPL.2017.12
- [39] Lucian Petrescu, Anca Morar, Florica Moldoveanu, and Alin Dragos Bogdan Moldoveanu. 2019. GPU Sparse Ray-Traced Segmentation. *IEEE Access* 7 (2019), 68511–68521. <https://api.semanticscholar.org/CorpusID:174818776>
- [40] RenderKit contributors. 2026. Embree Ray Tracing Kernels. <https://github.com/RenderKit/embree>.
- [41] Justin Salmon and Simon McIntosh-Smith. 2019. Exploiting Hardware-Accelerated Ray Tracing for Monte Carlo Particle Transport with OpenMC. *2019 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)* (2019), 19–29.
- [42] Michael Schwarz and Hans-Peter Seidel. 2010. Fast Parallel Surface and Solid Voxelization on GPUs. *ACM SIGGRAPH Asia 2010 Papers* (2010).
- [43] Xuri Shi, Kai Zhang, X. Sean Wang, Xiaodong Zhang, and Rubao Lee. 2025. RayDB: Building Databases with Ray Tracing Cores. *Proceedings of the VLDB Endowment* 19, 1 (2025), 43–55. doi:10.14778/3772181.3772185
- [44] shocker-0x15. 2026. OptiX Utility: OptiX 9 Lightweight Wrapper Library. [https://github.com/shocker-0x15/OptiX\\_Utility](https://github.com/shocker-0x15/OptiX_Utility). Typed payload and attribute signatures, automatic packing, usage annotations, and host-side object management.
- [45] Slang Project. 2026. Capabilities—Slang User’s Guide. <https://shader-slang.org/slang/user-guide/capabilities>.
- [46] Robert E. Strom and Shaula Yemini. 1986. Tpestate: A Programming Language Concept for Enhancing Software Reliability. *IEEE Transactions on Software Engineering* SE-12, 1 (1986), 157–171. doi:10.1109/TSE.1986.6312929
- [47] Sizhe Sui, Luis Sentis, and Andrew Bylard. 2024. Hardware-Accelerated Ray Tracing for Discrete and Continuous Collision Detection on GPUs. *2025 IEEE International Conference on Robotics and Automation (ICRA)* (2024), 16133–16139.
- [48] Taichi developers. 2026. Taichi Documentation. <https://docs.taichi-lang.org/>. Parallel programming language and runtime.
- [49] Bin Wang, Ingo Wald, Nate Morrical, Will Usher, Lin Mu, Karsten Thompson, and Richard Hughes. 2022. An GPU-Accelerated Particle Tracking Method for Eulerian–Lagrangian Simulations Using Hardware Ray Tracing Cores. *Computer Physics Communications* 271 (2022), 108221. doi:10.1016/j.cpc.2021.108221
- [50] Ziming Wang, Kaiyin Zhang, Yangming Lv, Yinglong Wang, Zhigang Zhao, Zhenying He, Yanan Jing, and Xiaoyang Sean Wang. 2024. RTOD: Efficient Outlier Detection With Ray Tracing Cores. *IEEE Transactions on Knowledge and Data Engineering* 36 (2024), 9192–9204.
- [51] Zhixiong Xiao, Mengbai Xiao, Yuan Yuan, Dongxiao Yu, Rubao Lee, and Xiaodong Zhang. 2025. A Case Study for Ray Tracing Cores: Performance Insights with Breadth-First Search and Triangle Counting in Graphs. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 9, 2, Article 16 (2025), 25 pages. doi:10.1145/3727108

- [52] Stefan Zellmann, Martin Weier, and Ingo Wald. 2020. Accelerating Force-Directed Graph Drawing with RT Cores. In *2020 IEEE Visualization Conference (VIS)*. 96–100.
- [53] Hongrui Zhang, Yunan Zhang, and Hung-Wei Tseng. 2025. RTSpM-SpM: Harnessing Ray Tracing for Efficient Sparse Matrix Computations. *Proceedings of the 52nd Annual International Symposium on Computer Architecture* (2025).
- [54] Shiwei Zhao, Zhengshou Lai, and Jidong Zhao. 2023. Leveraging Ray Tracing Cores for Particle-Based Simulations on GPUs. *Internat. J. Numer. Methods Engrg.* 124, 3 (2023), 696–713.
- [55] Shaokun Zheng, Zhiqian Zhou, Xin Chen, Difei Yan, Chuyan Zhang, Yuefeng Geng, Yan Gu, and Kun Xu. 2022. LuisaRender: A High-Performance Rendering Framework with Layered and Unified Interfaces on Stream Architectures. *ACM Transactions on Graphics* 41, 6, Article 232 (2022), 19 pages. doi:10.1145/3550454.3555463
- [56] Yuhao Zhu. 2022. RTNN: Accelerating Neighbor Search Using Hardware Ray Tracing. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 76–89. doi:10.1145/3503221.3508409